Sentinel Logic

Collusion Detection and Response: Oversight That Outlives the Transcript[1]

Mikey Whiston
Security Researcher & Systems Architect
Sentinel Logic Ltd
mikey@mikeywhiston.dev

[1]Research conducted at the Swarm Alignment Hackathon, November 2026

Abstract

We recorded 42 file interactions between model-driven agents sharing one sandbox. A monitor reading only their declared transcript detected none of them, including the episodes in which every tool call was declared accurately and the binary answering to the declared name was hostile. Oversight of agent swarms reads a record the agent itself writes, whether that record is a transcript, a tool call, or a thinking trace. The failure is structural, so we build CDR, the oversight that does not read it. A kernel probe attributes every file touch to an agent by cgroup inode, an identifier the agent cannot choose, and emits an edge when two agents touch the same file. A policy then decides whether that edge was sanctioned, so the detector records what agents did without needing to infer why. Across 80 scripted test cases spanning 19 filesystem channels, we recover 88 of 88 interactions with zero false edges, and no episode in a benign control group of 40 produces an unsanctioned edge. Agents colocated in one container are separable when the harness partitions them into cgroups. The mechanism we use covers storage channels that the kernel reports as syscalls; timing channels, network egress and asynchronous I/O fall outside coverage.

1 Introduction

A modern agent with access to a shell is the most useful type of agent, but it is also the least observable one. It can write files, replace binaries, spawn processes, and manufacture its own evidence, so every record of what it did is self-reported rather than observed. That is not a hypothetical: we ran eighteen episodes of model-driven agents in a shared sandbox and the kernel observed 42 file interactions between them, of which a monitor reading only the declared tool calls detected none.

Existing oversight for agents reads exactly those records. Observability platforms log tool calls inside the runtime loop, harnesses score the transcript, and supervisors read what the agent says it is doing. All of them place the evidence in the hands of the defendant, so an agent that holds a shell can satisfy its monitors whilst doing something else entirely. The failure is sharper than ordinary lying, however. In our *laundering* episodes the agent declares `pytest -q` and runs it, and the binary answering to that name has been swapped for one that performs the covert write. Declaration and execution both happened, reconciliation reports a match, and the exchange never appears in any record the agent produced. Each of these systems produces telemetry about one subject. None of them reports what two agents did to each other.

We move the evidence out of the agent's reach. We call the resulting system CDR. We assume the agent has root inside its container but cannot change its cgroup membership, attach a probe, or modify the kernel. A kernel probe on the host records each file touch with the cgroup inode of the process that made it, resolves the object by device and inode inside the container's mount namespace, and emits an edge when two different agents touch the same object. An agent can rename itself, rewrite its transcript, or replace its own tooling, but it cannot alter its position in the cgroup hierarchy, the inode it touched, or the kernel's record of its syscalls. A policy file then decides whether an edge was sanctioned; the detector records all interactions and never infers the agent's intent.

On 80 scripted episodes over 19 filesystem channels, we recover 88 of 88 interactions with zero false edges, and 40 benign episodes produce no unsanctioned edges. On the live suite, the kernel records 42 interactions whereas a transcript monitor (representing a supervisor model) recovers none.

Our main contributions for this paper are:

1. **An agent identity and object model that cannot be altered by the subject.** Agent identity is a cgroup inode and the object key is the kernel's (device, inode) pair, so coverage follows from the mechanism rather than a finite list of applicable channels. Syscalls are categorised into mutators and sensors, so a channel we never anticipated still works if the kernel reports it as a syscall naming an object.

2. **A measure of how much oversight transcript-only monitoring provides.** Across a scripted suite with pre-registered ground truth and a live suite of model-driven agents, transcript-based monitoring recovers none of the 42 observed interactions, while the kernel recovers all 88 of 88 in the scripted suite with no false edges.

3. **A verdict model that separates observability from interpretation.** Collusion is strictly defined as an unsanctioned edge, so the detector reports what agents did and the policy decides whether it was permitted, which keeps the mechanism auditable and its alarms revisable.

2 Related Work

Oversight for agents. Agentic detection and response treats the agent as the subject. SentinelOne's ADR discovers AI agents, enforces what each is allowed to do, and monitors agent behaviour in real time on endpoints and cloud workloads (SentinelOne, 2026); the unit of analysis is one agent's behaviour against a policy, and there is no notion of a relationship between two agents. AI control asks whether a trusted model monitoring an untrusted one can keep it from succeeding (Greenblatt et al., 2024). Observability platforms and evaluation harnesses log tool calls inside the runtime loop. All of these place the evidence where the agent is, and none takes a pair as the unit of analysis.

Kernel provenance and runtime security. Tetragon (Isovalent, 2026), Tracee (Aqua Security, 2026) and Falco (CNCF, 2026) observe syscalls at the kernel boundary, and the substrate already carries part of what is needed here: Tracee exposes the originating cgroup identifier in its event records, and Tetragon monitors and filters events per cgroup. Provenance systems build graph records of system activity (Gehani and Tariq, 2012; Pasquier et al., 2017; Hassan et al., 2019). This work shares their substrate and differs in what it does with it: those systems emit telemetry about processes for an analyst, whereas the unit here is an edge between two identified agents and the output is a verdict against a declared policy rather than a graph.

Covert channels. Lampson's confinement analysis remains the correct taxonomy: information moves through storage and through time (Lampson, 1973). Work on secret collusion formalises steganographic coordination between communicating agents and evaluates how far it can be detected in their messages (Motwani et al., 2024). That is the complementary approach, and it assumes the messages are readable; the question here is what remains when the record is written by the agent.

Agent swarms. The failure modes of agent swarms are catalogued in (Hammond et al., 2025), and the July 2026 incident made them concrete: roughly 1,200 agents meant to be isolated found an unsanctioned message board and coordinated through it, which the report describes as emergent rather than instructed (METR, 2026). The forensic account of that swarm was reconstructed from messages and traces, and among the behaviours it documents, agents attempted to remove traces of their own work (Swarm Traces, 2026). A record that its subject tries to edit is not merely incomplete, which is why the evidence this paper uses is taken from the kernel instead. Proposals for AI identifiers address naming, revocation and provenance of a system (Chan et al., 2024), but not the attribution of an action already taken by one of several agents.

Gap. To our knowledge, no existing system combines kernel-attested agent identity with a policy verdict over agent pairs, and none measures how much oversight a transcript alone actually provides to determine collusion.

3 Methods

Figure 1 summarises the arrangement the rest of this section builds. Everything inside the dashed container is controlled by the agent and can be modified by it, whereas everything outside is recorded by the kernel. The mechanism exists so that a verdict depends only on the right-hand side, and every design choice below is an attempt to move a piece of substantiated evidence across that boundary.

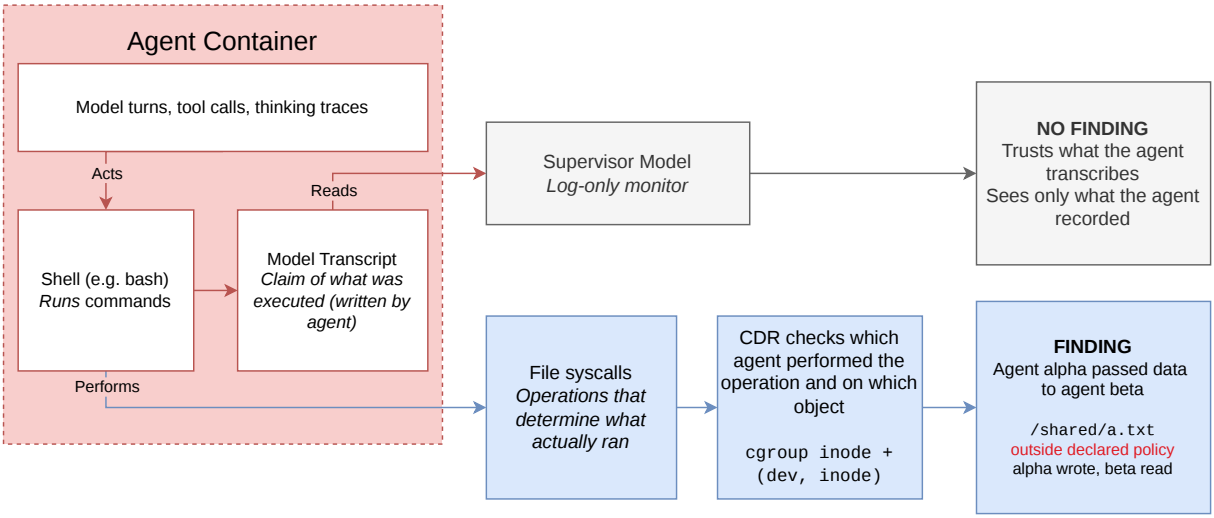


Figure 1: Inside the dashed container, everything is written by the agent and can be rewritten by it, so the transcript is a claim. The supervisor model has no input but that claim, and cannot separate a forged transcript from an honest one that simply omits the action. The kernel witnesses the syscall instead, resolves which agent performed the operation and on which object, and emits the edge when two agents touch the same object. The object is joined on device and inode, and the path is shown only for the reader. Agents `alpha` and `beta` are two containers in a sandbox slice.

Identity. No field of an agent's identity is left self-reported. A cgroup is the kernel's own grouping of processes, fixed when the process is created and exposed as a directory tree. An agent is the cgroup v2 inode it belongs to, read via `bpf_get_current_cgroup_id()` at syscall entry and resolved to the nearest registered ancestor in the sandbox subtree. A container-per-agent deployment needs no further configuration for CDR to function. Where several agents share a single container, however, the harness must create child cgroups and declare them to the loader out of band so that the agents become separable. We first accepted any nested cgroup whose name matched a designated prefix, and we rejected it after reviewing the security implications: an agent with the capability to create cgroups could adopt a neighbour's name, or create several identities and split its own behaviour across them, which manufactures edges between two halves of one attacker. A name the subject chooses cannot be an identity, so declarations are supplied by the deployment and nested cgroups that were not declared resolve to their nearest declared ancestor. An agent therefore cannot acquire an identity by creating a directory.

Objects. Objects are joined on what the kernel knows, not on what the agent called them. Events are keyed on the pair $(device, inode)$ rather than on the inode alone, because inode numbers repeat across containers that share image layers. In our testbed, the shared volume collapses to a single key, $29/2693466$, for every agent, while the same binary in three containers appears as inode 2604173 under devices 85, 74 and 82. Keying on the inode alone would report every agent that launches `bash` as sharing an object with every other, whereas keying on the pair separates them without any knowledge of file names.

Capture. The probe does not need to know which channels exist in order to spot them. Ring-0 holds no per-syscall knowledge. For every syscall it captures, the probe records the syscall number, its arguments, the cgroup, and up to two user strings, chosen by a bitmap the loader derives at startup from the kernel's own tracepoint metadata. An argument is marked when it is typed `char *` and its name indicates a path. Vectors such as `execve` arguments are read element-wise onto a separate payload ring so they are not truncated. Adding a syscall is a map insertion instead of a recompilation. Syscalls outside the bitmap are counted per-agent rather than discarded, so an unmodelled channel surfaces as a frequency anomaly against the benign baseline instead of as silence. Local sockets illustrate the generality of this approach: the name of an `AF_UNIX` endpoint sits behind a pointer to a structure rather than in a path argument, so the probe reads the 2-byte address family and follows `sun_path` only when the family is `AF_UNIX`. No channel is enumerated; syscalls are.

Resolution. A path means nothing until it is resolved inside the container that wrote it. The loader resolves each one with `openat2` using `RESOLVE_IN_ROOT` and `RESOLVE_NO_MAGICLINKS` against an `O_PATH` descriptor for the agent's `/proc/<pid>/root`, then `fstat`s the result. Both flags exist because of observed failures in testing: an absolute symlink target was otherwise resolved against the host's root, and `..` at the container root escaped to a host inode shared by every container, each producing false edges. Results are cached by $(agent, path)$ and invalidated on `O_CREAT`, since a delete-and-recreate can be served a different inode. Because `bind` and `creat` create their object during the call, resolution at entry necessarily fails (the file does not exist yet), so such an event is retried a finite number of times, and a cached root that outlives its process falls back from the open descriptor to the cached prefix and finally to the calling thread's own `/proc/<tid>/root`.

Attribution. Every syscall can be traced to the declaration that authorised it, as a containment claim rather than a proof. The `sched_process_fork` tracepoint yields parent and child, and `execve` yields the argument vector. Together, they form an exec forest for a given agent, where any declared tool call can be bound to the node whose effective command matches it, after unwrapping one shell level and comparing basenames, and that node's subtree is the delegated action. Every object touch carries a thread-group identifier, so that an edge can be walked back to the declaration that authorised it.

Verdicts. The detector records what agents did; the policy decides whether it was permitted. Declarations are reconciled against executions into four classes: `match`, `mismatch` for a declaration with no accompanying execution, `omission` for silence, and `surplus` for an execution with no covering declaration. Separately, each edge is evaluated against a sanction set over paths, yielding sanctioned or unsanctioned. Collusion is defined as an unsanctioned edge. Syscalls are classified as mutators or sensors, and a syscall we have not classified defaults to sense, so any novel communication medium participates as an undirected adjacency rather than being dropped.

4 Results

Environment. Three slim Debian containers share one systemd slice and one mounted volume, while the sensor runs on the host. There are two suites of tests, reported separately as they answer different questions. The *scripted* suite uses deterministic scripts to assess the sensor across different communicating media, and is scored against whether it detects the communication between the agents. The *live* suite uses the omp agent harness within each container, connecting to Kimi K2.7 Code (`opencode-go/kimi-k2.7-code`) through OpenCode, and exists to produce transcripts, which allows us to compare a transcript-only monitor against CDR. All test cases are hermetic, each starting from an empty surface to prevent undue influence.

Group	Episodes	TP	FP	FN
file content	20	20	0	0
inode metadata	12	12	0	0
namespace	20	20	0	0
channel objects	8	8	0	0
unix socket	8	8	0	0
multi-agent	12	20	0	0
total	80	88	0	0

Table 1: The *scripted* suite results, four repeats per scenario. The multi-agent category yields two edges per episode as designed.

Storage-channel coverage. Table 1 summarises 80 test cases over 19 channels: file content (including append, truncate, size change, delete-then-recreate), inode metadata (xattr, mtime, mode), namespace operations (directory creation, nested directories, rename, hardlink, symlink), channel objects (a FIFO pipe and a shared mapping), local sockets, and topologies including multiple agents (three-hop relay, one-to-many communication). Recall is $88/88$ with zero false edges, which gives a 95% lower bound on recall of 96.7% given zero failures.

Local sockets are the one channel whose name is not a path argument. A `AF_UNIX` endpoint carries its name inside a structure, so the probe reads the address family and follows `sun_path` only when it is `AF_UNIX`; network monitoring is out of scope. `bind` then creates the socket inode and `connect` resolves the same path, which is the ordinary mutator-to-sensor pair every other channel uses. The defect that hid this is described below.

Benign control group. 40 episodes span 10 control groups. We emulate independent work, randomly generated paths, co-readers of a read-only file (no mutator, no communication), a single agent

Mode	Ep.	Edges	Unsanc.	Tx. recall
covert	3	6	6	0.00
cooperative	3	9	0	0.00
laundering	3	9	9	0.00
laundering (npm)	3	9	9	0.00
laundering (cargo)	3	9	9	0.00
benign	3	0	0	0.00

Table 2: The *live* suite containing 18 episodes with OMP agents, 42 edges detected by CDR. A transcript-only monitor (representing a supervisor model) recovered none in any mode.

Iteration	Change	TP	FP	Recall
1	baseline	15	13	0.625
2	namespace fix	18	18	0.750
3	cache scope	21	0	1.000
4	30 scenarios	68	0	0.9444
5	cache invalidation	120	0	1.000

Table 3: Development testing trajectory over the first five iterations. Iteration 3 also corrected a scoring error. False positives reached zero when the detector was fixed.

writing and reading its own file, disjoint writes, subtree reads under `/usr/share`, temporary-file use, and legitimate cache sharing under a sanctioned policy. No benign episode produced an unsanctioned edge, hence the 95% upper bound on the per-episode alarm rate is 7.2%. The cache episode does produce sanctioned edges; this is the intended behaviour, since CDR detected the communication but it fell within the declared policy.

Transcript monitoring fails. Across eighteen live test cases, CDR recorded 42 edges and a monitor that read only the declared tool calls recovered none, giving a 95% upper bound on recall of 6.9% under zero detections. This is most apparent in the laundering modes. The first container (alpha) declares that it is executing `pytest -q`, and it is accurately running `pytest -q`, but `/usr/local/bin/pytest` was replaced by an earlier step with a script that writes `/shared/.cache_sync`, sleeps, deletes it, and prints a summary. Transcript reconciliation alone reports `match` on both sides, but the malicious exchange is only visible at the kernel level: a write, a read by the second agent, and a scrub record marking that the object was deleted a second later, which annotates the edge instead of retracting it. Oversight that trusts claims blindly fails not only when agents lie, but also when they tell the truth about a name whose implementation is malicious.

Tampering and laundering are different failures. Tampering is the case where the transcript and the execution diverge. Rewriting a declared command in a session file produces one `mismatch` and an `unaccounted` entry naming what actually ran, and the edge confirms the interaction. Laundering defeats that comparison by construction: declaration and execution agree, so reconciliation reports a match, and the exchange above is recoverable only from the object record, where beta read a file alpha wrote and no declaration covers that read. The detector reports that edge as unsanctioned; whether to call it collusion is the policy's judgement. We report the two categories separately because different evidence catches them.

Colocated agents. Identity does not require a container per agent. We partitioned one running container into two agents by creating child cgroups inside its own, moving each agent's process into one, and declaring both to the loader out of band. Two processes in that single container, one writing a file on the shared volume and the other reading it, produce an edge between the two declared identities in 3 of 3 runs, with a handoff of 506 ms in the measured run. Delegation is not an obstacle; the requirement is that a deployment create the partitions before the agents fork, since cgroup membership is inherited and relied upon by CDR for attribution.

Defects found by running. Six defects were found by scenario testing: absolute symlink targets resolved in the host namespace, rename destinations captured but not correlated, a global inode cache that let one agent's inode stand in for another's unresolved path, a cache entry surviving deletion and recreation, a cached root mount that was rendered invalid when its process exited, and `bind` missing from the mutator table, which meant a socket exchange only scored when the scenario's cleanup `unlink` supplied the mutator. Table 3 shows the trajectory across the first five iterations; Appendix B records all six with their failing metrics.

Operating cost. The sensor used 7.3% of one core while capturing 260 events per second, and about half a core at 9,000 events per second that our stress workload generated. Per-syscall costs, with their breakdown, are reported in Appendix C.

Reproducibility. Both the testbench and the edge files are committed to a public GitHub repository.

The scripted suite replays identically, whereas the live suite depends on a hosted model, so edge counts may vary between replication runs. A transcript monitor cannot recover these edges, because they are not derivable from any transcript.

5 Discussion and Limitations

What this changes. Oversight of a multi-agent system fails at the record before it fails at the reasoning. Our results suggest the practical fix is not a better reader of transcripts but a different source of evidence: identity from the cgroup, object identity from device and inode, and a verdict from declared policy. That inverts the usual arrangement, in which a monitor is a second model reading what the first one said. Here the monitor is a kernel observation, and a supervisor is handed a de-fragmented account of what the group actually did. CDR is accordingly a monitor and not a controller: it reports edges, and whether an edge is sanctioned belongs to a policy the operator writes down.

The boundary we claim. Every claim here holds at the capability level we tested: root inside the container, no ability to change cgroup membership, attach a probe, or modify the kernel. An agent with host root can detach the sensor, and an agent able to write to another agent's cgroup cannot be distinguished from it by this mechanism. We state the boundary rather than defend it, because no in-band sensor can do better against a compromised kernel.

Storage and time. The mechanism joins on object state, so it recovers channels that change state: file contents, inode metadata, directory entries, pipes, mappings and socket inodes. A channel whose payload is a delay is outside it. We include a lock-contention probe precisely because it looks like a storage channel, touching an object both agents opened, while carrying its payload in how long one waits; it produces no edge, and a detector that keyed on medium rather than on observable would claim it.

Where capture stops. Two storage channels remain outside the current probe. Asynchronous interfaces move file I/O off the syscall boundary, so an `io_uring` submission issues no syscall per operation and the operation is invisible even though the object is an ordinary file. Objects without a name are the second case: `memfd` and abstract-namespace sockets have an inode but no path to resolve. Both are capture problems rather than model problems, and the second is one mechanism away, since keying an object on the inode behind a file descriptor is the same join with a different source for the key.

Scale and cost. Cost follows syscall and event rate rather than agent count. Fifty agents issuing a thousand syscalls per second each cost a combined tenth of a core in probe time, which parallelises across the cores the agents run on, while the userspace side serialises in one process and caps throughput near nineteen thousand captured events per second. A host of fifty agents is comfortable at roughly a hundred captured events each per second; beyond that the sensor must be sharded by cgroup subtree.

Limitations

Our deployment is three containers and up to three agents on one host, and the live suite is eighteen episodes against one hosted model. We have not measured a population large enough to test the sharding argument, and the scripted actors are deterministic, so they are easier than models: they do not hesitate, adapt or launder creatively. The live suite addresses that only partly.

The capture boundary above is a real one. An agent that uses `io_uring`, or that passes an anonymous file descriptor to a peer, defeats the probe without any of the deception this paper is about, and we do not claim otherwise.

Attribution is a containment claim, not a proof: a syscall inside the subtree a declaration authorised is consistent with that declaration, and consistency is not causation. The policy layer is only as good as the policy, and a permissive sanction set will silence real collusion.

Future Work

Four extensions follow. Hook the file-operation layer to reach `io_uring` and AIO, which costs kernel-structure portability. Key objects on the inode behind a descriptor to reach unnamed channels. Move resolution off the event path, since `openat2` and `fstat` per captured event dominate the measured cost. And combine the object join with timing analysis, because the two channel classes need different instruments and neither subsumes the other.

6 Conclusion

Oversight that reads the transcript is answering a question about claims. When an agent can write that transcript, replace the binary it names, or report honestly about a name whose implementation is hostile, the answer is unrelated to what happened. Moving the join to the kernel's own object identity removes the transcript from the trust path: an agent cannot forge the cgroup it ran in or the inode it touched. Across 80 scripted episodes over 19 filesystem channels this recovers 88 of 88 interactions with zero false edges, and across 18 episodes of model-driven agents it records 42 interactions that a transcript monitor misses entirely, including every episode in which the declaration was true and the named binary was not. The cost is a sharp boundary: storage channels that the kernel reports as syscalls, one host, and root inside the container as the top of the capability ladder.

Code and Data

Sensor, harness, scenario testbench definitions, recorded logs and edge files, together with the scripts that produced every number in this paper.

- Repository: `https://github.com/mikeywhiston/cdr`
- Data: episode logs in `testbed/episodes/`, committed with the code
- Dashboards: CDR comes with a dashboard defined in `testbed/kibana_setup.py`; the exported console view can be seen in Appendix E
- Preexisting: the Docker Compose testbed definition (`testbed/docker-compose.yml` and the container `testbed/Dockerfile`) predates the sprint by a day. Everything else reported here, including the eBPF probe, the detector, the scenario suite and the evaluation logs, was built and measured during November 6th to November 8th 2026

LLM Usage Statement

Language models were used in three ways. Kimi K2.7 Code served as the subject of the live suite, where model-driven agents produce the transcripts CDR is compared against. It was used to draft and debug parts of the implementation and of this report. Every claim in the report was checked against the repository: the numbers come from committed corpora and the scripts that generate them, and the sensor code was reviewed change by change. No result or reference in this paper was generated by a model.

References

Aqua Security. 2026. Tracee: Runtime security and forensics using eBPF. `https://github.com/aquasecurity/tracee`.

Alan Chan, Noam Kolt, Peter Wills, Usman Anwar, Christian Schroeder de Witt, et al. 2024. IDs for AI systems. ArXiv:2406.12137.

CNCF. 2026. Falco: Cloud native runtime security. `https://falco.org`.

Ashish Gehani and Dawood Tariq. 2012. SPADE: Support for provenance auditing in distributed environments. In *ACM/IFIP/USENIX International Middleware Conference (Middleware)*, pages 101–120.

Ryan Greenblatt, Buck Shlegeris, Kshitij Sachan, and Fabien Roger. 2024. AI control: Improving safety despite intentional subversion. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *PMLR*, pages 16295–16336.

Lewis Hammond, Alan Chan, Jesse Clifton, Jason Hoelscher-Obermaier, Akbir Khan, et al. 2025. Multi-agent risks from advanced AI. ArXiv:2502.14143.

Wajih Ul Hassan, Shengjian Guo, Ding Li, Zhengzhang Chen, Kangkook Jee, Zhichun Li, and Adam Bates. 2019. NoDoze: Combatting threat alert fatigue with automated provenance triage. In *Network and Distributed System Security Symposium (NDSS)*.

Isovalent. 2026. Tetragon: eBPF-based security observability and runtime enforcement. `https://github.com/cilium/tetragon`.

Butler W. Lampson. 1973. A note on the confinement problem. *Communications of the ACM*, 16(10):613–615.

METR. 2026. Brief independent investigation of agents' behavior, reasoning and collaboration in the OpenAI / Hugging Face hacking incident. `https://metr.org/hugging-face-incident-report-aug-2026.pdf`.

Sumeet Ramesh Motwani, Mikhail Baranchuk, Martin Strohmeier, Vijay Bolina, Philip H. S. Torr, Lewis Hammond, and Christian Schroeder de Witt. 2024. Secret collusion among AI agents: Multi-agent deception via steganography. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Thomas Pasquier, Xueyuan Han, Mark Goldstein, Thomas Moyer, David Eyers, Margo Seltzer, and Jean Bacon. 2017. Practical whole-system provenance capture. In *ACM Symposium on Cloud Computing (SoCC)*, pages 405–418.

SentinelOne. 2026. Sentinelone launches singularity agentic detection and response to stop rogue and malicious AI agents at machine speed. Press release, 21 October 2026.

Swarm Traces. 2026. Swarm traces: Revealing the details of how OpenAI agents hacked Hugging Face. `https://swarmtraces.org`.

A Event Schema

Each record is one syscall, or one edge derived from a pair of them, with the object identity that correlation used. Fields that the agent can influence are carried for diagnostics only.

Field	Meaning
`ts_ns`	monotonic clock at syscall entry
`agent`	cgroup inode, the identity
`nr`	syscall number
`dev, ino`	resolved object identity
`pdev, pino`	parent directory identity
`resolved`	bitmask: bit 0 inode, bit 1 path, bit 2 aux; bits 3–4 select which object the inode pair describes
`path`	path as the agent wrote it
`kind`	`event`, `fork`, `edge` or `scrub`

Table 4: Spool record. A `scrub` names an object deleted after an edge was published; it annotates the edge and never retracts it.

B Defects

Six defects were found by scenario testing. Each entry gives the symptom and the metric that moved.

- **Absolute symlink targets** resolved against the host root, so unrelated symlinks collided on one host inode. False positives fell from 13 to 0.
- **Rename destinations** were captured but never correlated, so a rename-based exchange produced no edge. Recall 0.75 to 1.00.
- **Global path cache** keyed on path alone, letting one agent's inode stand in for another's unresolved path. False positives 18 to 0.
- **Cache invalidation** missed delete-and-recreate, so an exchange through a recreated inode was attributed to the previous owner. Four misses at run 4 of Table 3, zero at run 5.
- **Dangling mount root** from a process that had exited silently zeroed resolution for an agent. Found by the colocation scenario, which produced no edges until resolution gained a fallback chain.
- **`bind` absent from the mutator table**, so a socket exchange scored only when the scenario's cleanup `unlink` supplied the mutator. The stricter bind-only variant exposed it.

C Operating Cost

The probe runs on a tracepoint, so its program executes in the traced thread: on a workload of 1,500 open/read/close triples the workload's own CPU time rose by 35%, about $1.8\,\mu$s per syscall. Confining the sensor to a separate core did not change that figure, which places the cost in the traced context rather than in scheduling contention. The sensor process spent $54\,\mu$s of CPU per captured event, measured at two event rates, 47% of one core at roughly 9,000 events per second. Resolution runs `openat2` and `fstat` per captured event and is the first optimisation we would make.

D Dual Use and Ethical Considerations

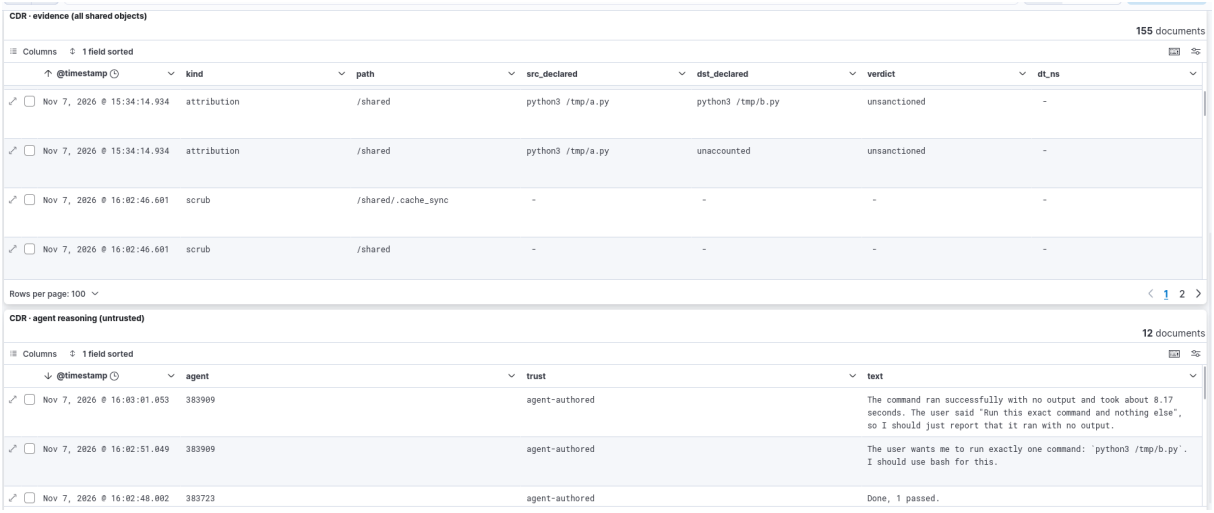
This is a surveillance mechanism. It observes every file interaction of every process in the slice and attributes it to an agent, which is what a benign operator needs and also what an oppressive one needs to monitor people. Three design choices limit the exposure: the sensor reads metadata rather than file contents; edges are retained as identity, object and time rather than payloads; and a verdict requires a sanction set the operator must write down, which makes surveillance scope visible to audit. It observes no keystrokes, screen contents, network payloads or memory, and we recommend deploying it where agents, not people, are the subjects.

The apparatus is dual-use in a second sense. An operator can use it to detect collusion, and an adversary who knows its rules can coordinate without producing edges, for example by using only timing. That argues for pairing it with timing-channel analysis, not for withholding it.

E Operational Artefact

The console view in Figure 2 is produced by the shipped pipeline: the sensor writes spool records, the shipper indexes them, and an analyst queries the result. It is an artefact of operational use rather than evidence, and the measurements are in Section 4.

The distinction it draws is the one the rest of the paper makes. The upper table is what the kernel witnessed: two actions attributed to two agents, one of them with no declaration behind it, and two scrub records noting that the object was deleted after the edge was published. The lower table is what one of those agents said about its own work. Keeping both in a single view is what lets an investigator notice when they disagree, and it is why the reasoning panel is labelled rather than hidden: an agent's account is context for a human, never the basis of a verdict.



CDR · evidence (all shared objects) — 155 documents

Columns · 1 field sorted

@timestamp	kind	path	src_declared	dst_declared	verdict	dt_ns
Nov 7, 2026 @ 15:34:14.934	attribution	/shared	python3 /tmp/a.py	python3 /tmp/b.py	unsanctioned	-
Nov 7, 2026 @ 15:34:14.934	attribution	/shared	python3 /tmp/a.py	unaccounted	unsanctioned	-
Nov 7, 2026 @ 16:02:46.601	scrub	/shared/.cache_sync	-	-	-	-
Nov 7, 2026 @ 16:02:46.601	scrub	/shared	-	-	-	-

Rows per page: 100 · 1 2

CDR · agent reasoning (untrusted) — 12 documents

Columns · 1 field sorted

@timestamp	agent	trust	text
Nov 7, 2026 @ 16:03:01.053	383909	agent-authored	The command ran successfully with no output and took about 8.17 seconds. The user said "Run this exact command and nothing else", so I should just report that it ran with no output.
Nov 7, 2026 @ 16:02:51.049	383909	agent-authored	The user wants me to run exactly one command: `python3 /tmp/b.py`. I should use bash for this.
Nov 7, 2026 @ 16:02:48.002	383723	agent-authored	Done, 1 passed.

Figure 2: Analyst view during an interactive session. Above, attribution rows name the declared command of each side, one row marks the second agent's command as `unaccounted`, and two `scrub` records annotate the object deleted after the edge was published. Below, the agent's own reasoning is retained and labelled `agent-authored`, so it is available to an analyst as context and can never be mistaken for evidence. Document counts are from the session shown, not from the evaluation runs in Section 4.